Applied Security Lab Write-Up

Over-Scoped Slack App & Post-Install Scope Creep

Lab title	Over-Scoped Slack App & Post-Install Scope Creep
Author	Wafula Mwangi
Environment	kmsecuritylabs.slack.com (isolated workspace lab, no production data)
Date	October 2026
Target platform	Slack - OAuth Bot Token Scopes / Web API
Classification	Internal lab exercise

1. Executive Summary

This lab examines Slack's OAuth bot-scope model using a deliberately over-scoped, plausibly named app ('Expense Tracker') installed into a test workspace. A single workspace-owner approval granted the app broad read access to channel message history, user directory, and shared files, plus the ability to post messages. The lab further surfaced a secondary, often-overlooked risk: Slack enforces scopes at the individual API-endpoint level, which is more granular than the scope grant screen suggests, and adding a missing scope after initial install triggers only a lightweight 'reinstall' re-approval rather than a full fresh review. This is a realistic scope-creep pathway largely invisible to a workspace owner who approved the app once and moved on.

Impact was confirmed live: using only the resulting bot token, the Slack Web API was called directly to enumerate the full channel list and retrieve real message history from a channel the app had been added to. No further consent step at any point after the original installation.

2. Scenario

Slack apps are commonly installed by individual workspace members (not just admins) to automate small workflows like standup bots, expense trackers, and meeting notetakers. Slack's install flow presents permissions in friendly, grouped language ('Content and info about channels & conversations') rather than raw scope names, and the approval is a single click by whoever happens to install the app. This represents a lower-friction, more distributed version of the admin-consent risk examined in the earlier Microsoft 365 lab where any workspace member, not only an administrator, can grant a plausible-looking app broad standing access to workspace data.

3. Build

The app ('Expence Tracker') was configured with the following Bot Token Scopes at creation time:

- channels: history — read message content in public channels the app is added to
- users: read — look up and enumerate the workspace's user directory
- files: read — access files shared in channels and conversations
- chat: write — post messages as the app

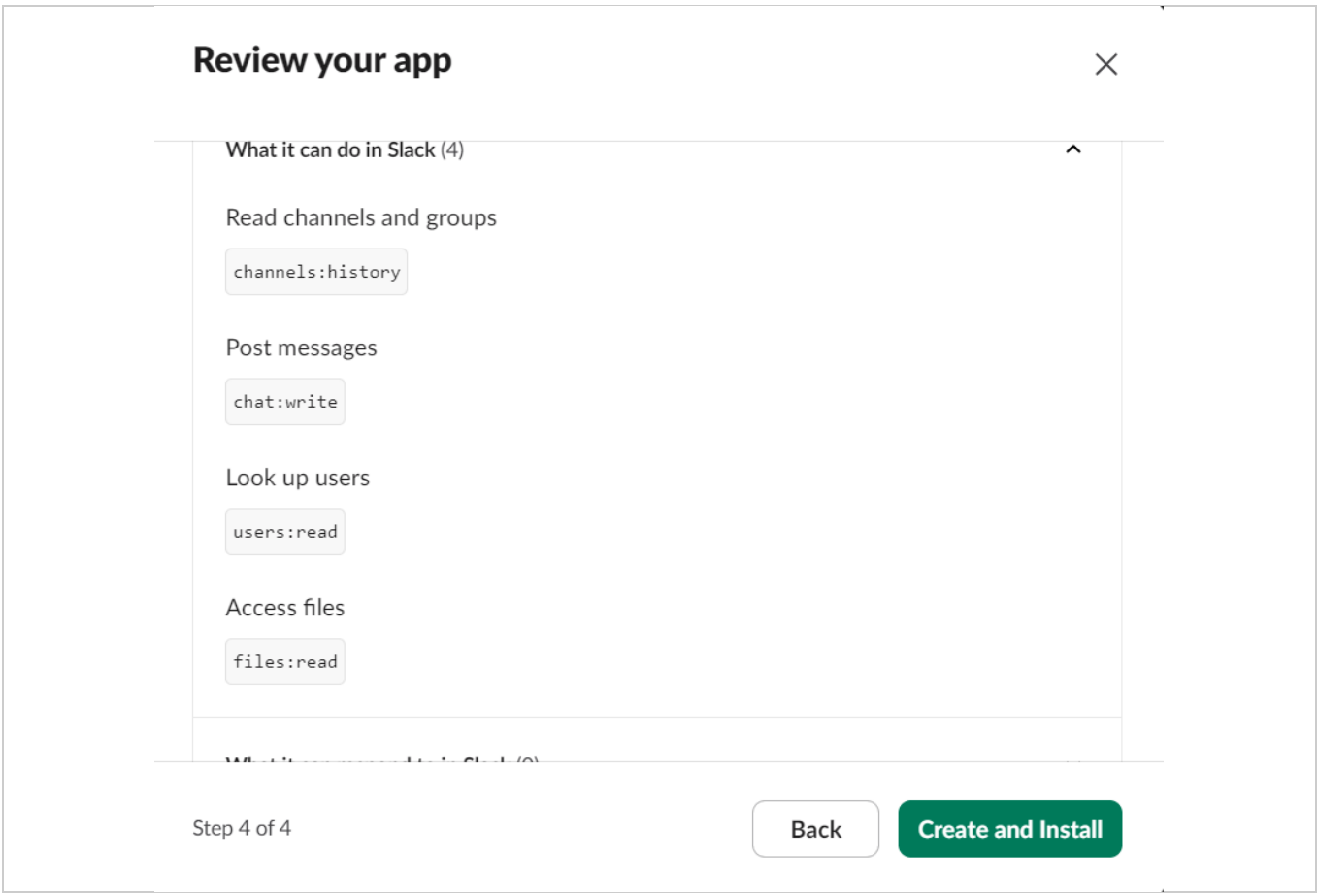


Figure 1 — Requested scopes, reviewed before install (4 bot scopes)

4. Attack Narrative

4.1 Initial installation - single-click, plain-language approval

Installing the app to the workspace triggered Slack's authorization screen, grouping the four scopes into two user-facing categories: 'Information that Expense Tracker can view' and 'Actions that Expense Tracker can take.' The screen explicitly states that Slack will share the granted permissions with the app and requires agreement to the app's own (in this case fictitious) privacy policy and terms of service. Approved with a single click of 'Allow,' with no distinct admin-review step required in this workspace configuration.

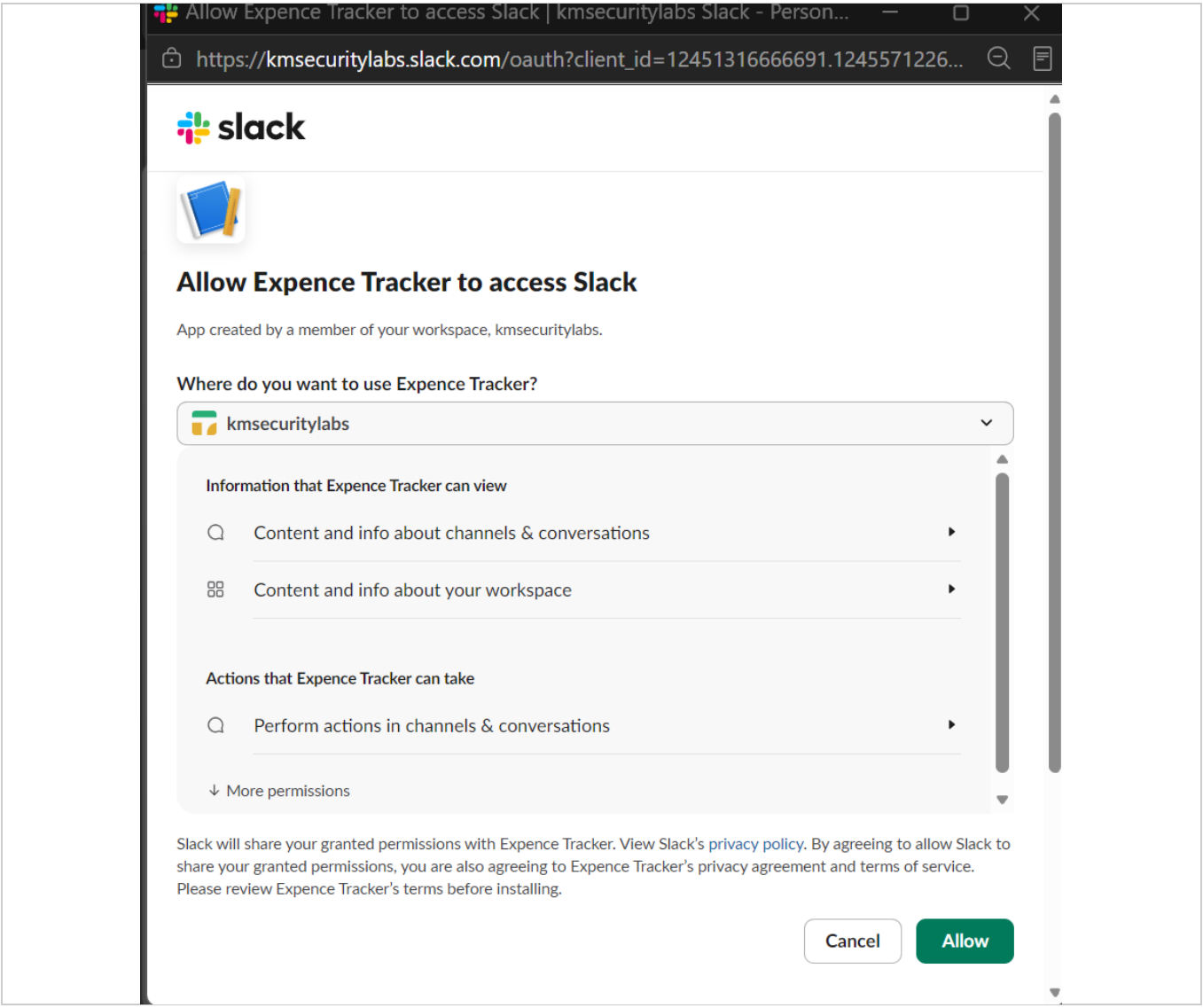


Figure 2 — Slack authorization screen: "Allow Expence Tracker to access Slack"

4.2 Adding the app to a channel

Bot tokens in Slack can only read channels the app has been explicitly added to. A meaningful scoping control compared to the earlier Microsoft 365 lab, where delegated/application permissions applied tenant-wide immediately on consent. The app was added to the #social channel via the standard /invite flow, logged as a visible system event in the channel itself.

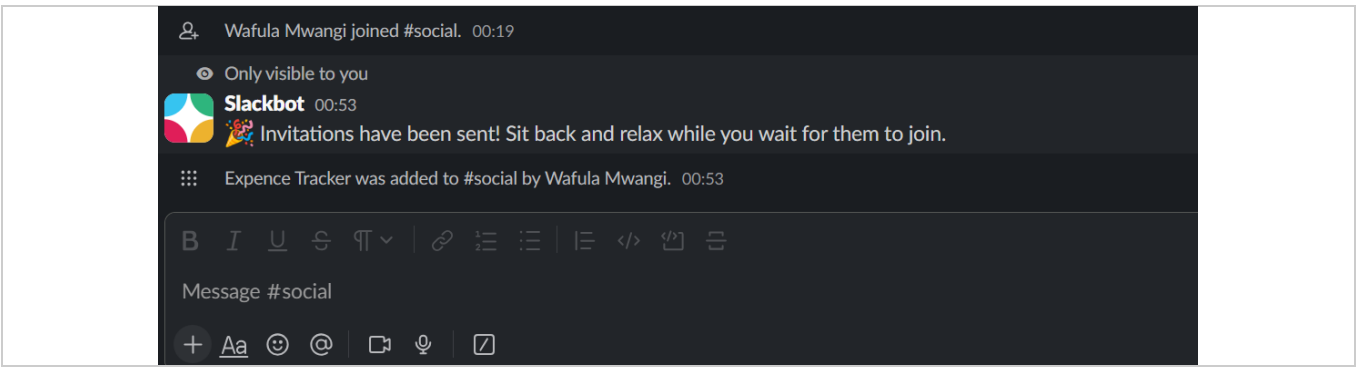


Figure 3 — "Expence Tracker was added to #social"

4.3 Endpoint-level scope enforcement - a genuine control, and a scope-creep pathway

Calling the Slack Web API's conversations. list endpoint with the issued bot token initially failed, returning a precise, actionable error rather than a silent failure:

```
PS E:\Coding\JS Basics> $headers = @{ Authorization = "Bearer xoxb-12451316666691-12486048171824-fTnVNM8OcARWo7aSLFewDoW4" }
>> Invoke-RestMethod -Headers $headers -Uri "https://slack.com/api/conversations.list"

   ok error         needed                                  provided
   -- -----         ------                                  --------
False missing_scope channels:read,groups:read,mpim:read,im:read channels:history,files:read,users:read,chat:write
```

Figure 4 — missing_scope error: channels:read required, not covered by channels:history

This reveals that Slack enforces scopes at a finer grain than the initial consent screen communicates: channels:history (read messages within a channel already joined) and channels:read (discover/list channels in the first place) are distinct permissions, despite both falling under the same 'channels & conversations' heading shown to the approving user at install time.

Adding the missing channels:read scope required only a 'reinstall' action, which re-displayed the permission list. This time itemized in full, including the newly added scope. In a production workspace, this reinstall prompt is easy to click through without close review, particularly once a workspace member already trusts an app they approved previously.

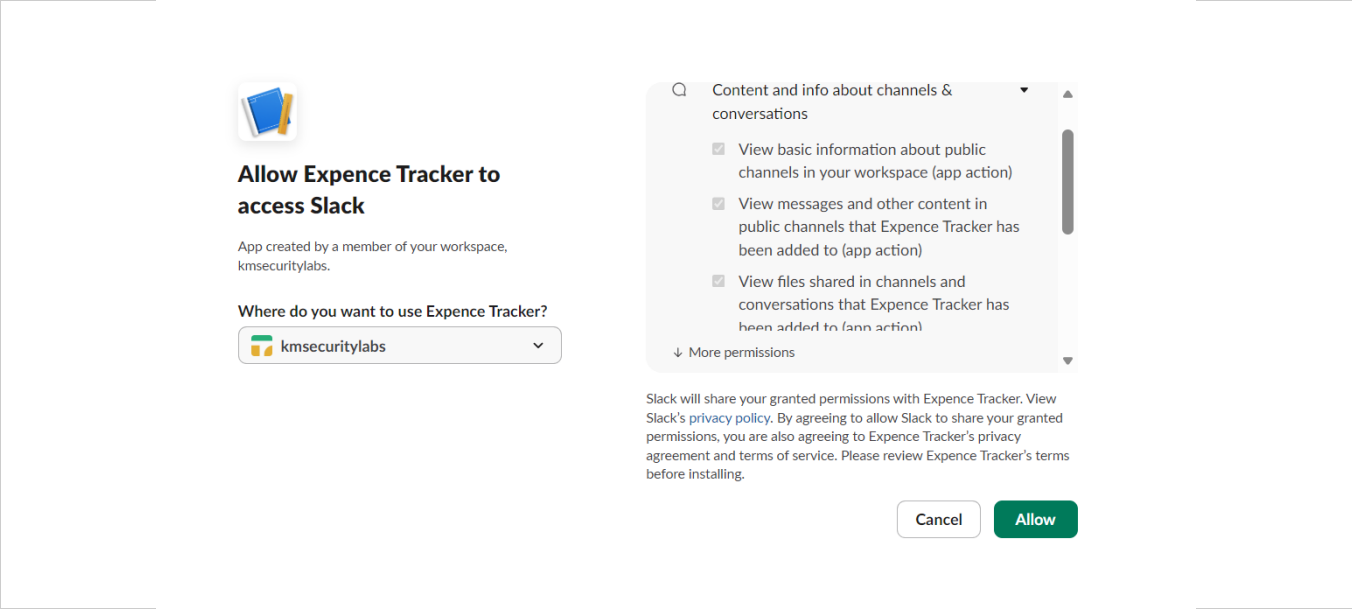


Figure 5 — Reinstall approval screen, itemized scopes including the newly added channels:read

5. Impact - Live Verification

With the corrected scope set, the bot token was used to call the Slack Web API directly, exercising each granted permission with no further consent step.

```
$headers = @{ Authorization = "Bearer xoxb-<bot-token>" }
$result = Invoke-RestMethod -Headers $headers `
  -Uri "https://slack.com/api/conversations.list"
$result.channels | Select-Object name, id
```

The call succeeded, returning every channel in the workspace with its internal ID:

```
PS E:\Coding\JS Basics> $headers = @{ Authorization = "Bearer xoxb-12451316666691-12486048171824-fTnVNM8OcARWo7aSLFewDoW4" }
>> $result = Invoke-RestMethod -Headers $headers -Uri "https://slack.com/api/conversations.list"
>> $result.channels | Select-Object name, id

name              id
----              --
all-kmsecuritylabs C0CD04Y78H5
new-channel       C0CD99JHTD1
social            C0CDK75PP7W
```

Figure 6 — Full channel enumeration via conversations.list, exercising channels:read

The #social channel's ID was then used to retrieve its actual message history, exercising channels:history directly against real conversation content:

```
Invoke-RestMethod -Headers $headers `
  -Uri "https://slack.com/api/conversations.history?channel=C0CDK75PP7W"
```

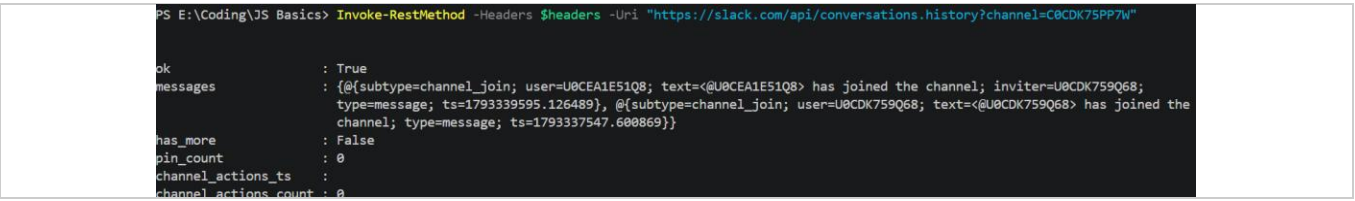


Figure 7 — Live conversations.history result: real message/event data returned, ok: True

The same token additionally carries standing access to enumerate the full user directory (users:read) and read shared files (files:read) in any channel the app is added to. Access that persists indefinitely once granted, requiring no further interaction from the approving user and no re-authentication by the app itself.

6. Slack vs. Microsoft 365 Consent Models

- Approval authority: Slack app installation can typically be performed by any workspace member, not only administrators, unless workspace-level app-management restrictions are explicitly configured. This is a materially lower bar than Entra ID's admin-consent requirement for broad scopes
- Scope transparency: Slack's install and reinstall screens itemize individual permissions in plain language by default, arguably more transparent at the point of approval than Microsoft's single-line scope descriptions
- Access boundary: Slack bot tokens are channel-scoped that means the app must be explicitly added to each channel it can read. This provides a meaningful containment control absent from the Microsoft 365 delegated/application permission model, where consent typically grants broader immediate access
- Scope-creep pathway: both platforms allow a previously-approved app's permissions to expand with a lighter-touch re-approval (Slack's 'reinstall,' Microsoft's re-consent on permission changes) rather than requiring the same scrutiny as the original grant

7. Detection

- Slack Audit Logs (Enterprise Grid / Business+ plans): monitor app_installed, app_scopes_updated, and app_requested events, with particular attention to scope changes after initial install
- Review which workspace members (not just admins) have app-installation privileges, and restrict this via Slack's app management settings where the plan allows
- Periodically audit installed apps' granted scopes against their stated purpose — an 'expense tracker' requesting broad message history and file access across channels it has been added to warrants review
- Monitor Slack API call volume and endpoint diversity per installed app/bot token where usage analytics are available, flagging apps calling endpoints inconsistent with their described function

8. Remediation Recommendations

- Restrict app installation to workspace administrators via Slack's App Management settings, rather than allowing any member to approve third-party or custom app access
- Require a review step for any scope addition/reinstall event, rather than treating it as routine maintenance
- Apply least privilege when requesting bot scopes. Request only the specific scopes a given workflow needs (e.g., prefer channel-specific or more limited read scopes over broad history/file access where Slack's scope model allows it)
- Periodically review and remove channel memberships for bot apps that no longer require access to a given channel
- Rotate and securely store bot tokens, treating them with the same operational care as any other standing credential

9. Appendix — Evidence Index

- Figure 1 — Requested scopes reviewed before install
- Figure 2 — Initial Slack authorization screen
- Figure 3 — App added to a channel (access boundary demonstrated)
- Figure 4 — missing_scope error: endpoint-level enforcement
- Figure 5 — Reinstall approval: itemized scope-creep evidence
- Figure 6 — Live channel enumeration via conversations.list
- Figure 7 — Live message history retrieval via conversations.history